MEGN 651A: Homework One

Rachel Bertaud

January 30th, 2026

The code used to produce this work can be located at this address:

https://github.com/rachbertaud/Advanced_CFD/tree/main/01_finite_difference_convergence.

For reproduction of results, simply download the folder `01_finite_difference_convergence`, call `$ make HW1`, and then run `plot_error.m` in MATLAB to produce the error plot, which will save automatically to your current directory.

Overview of Code

The purpose of this code is to solve for

$$g(x) = \frac{df}{dx}$$

given discrete points of $f(x)$, which we will define as

$$f(x) = \exp\left(\sin\left(\frac{2\pi x}{L}\right)\right).$$

Note that we can analytically solve for $g(x)$ and use this to verify our code behaves properly. We will assume periodic boundary conditions.

The code produced defines the user-input as `stop`, which determines the number of times we will run the code. We then loop through different number of discrete points of $f(x)$, N, which is changed by `stop` as follows:

```
for N = 16; while N < (16*(2**(stop - 1)) + 1); step by multiplying N by 2){
    //run main program
}
```

All manipulated variables are defined as doubles besides N and π. We define $h = L/N$, where $h = \Delta x$ and L is the length of our domain, which will be 2π for all N in this case. We then compute a numerical estimate of $g(x)$ using the centered finite-difference method, which is defined as:

$$g_{\text{numerical}}(x) = \frac{df_i}{dx} = \frac{f_{i+1} - f_{i-1}}{2h}.$$

Here is the for-loop pseudocode for how the boundary conditions are managed:

```
for all grid points
    if at first grid point
        g_numerical at first grid point = (f at second grid point - f at last grid point)/(2*h);

    else if at last grid point
        g_numerical at last grid point = (f at first grid point - f at last grid point)/(2*h);

    else
        g_numerical at current grid point = (f to the right - f to the left)/(2*h);
```

Error Analysis

To verify the code, error of the numerical solution is calculated as relative error using the infinity norm, which is defined as

$$E_\infty = \frac{||g_{\text{numerical}} - g_{\text{analytical}}||_\infty}{||g_{\text{analytical}}||_\infty} = \frac{\max|g_{\text{numerical}} - g_{\text{analytical}}|}{\max|g_{\text{analytical}}|}.$$

Here is pseudocode for how this code calculates this error:

```
max_numerator = 0
max_denom = 0
for all indicies
    if abs(numerical g_i - analytical g_i) > max_numerator
        max_numerator = abs(numerical g_i - analytical g_i)
    if abs(analytical g_i) > max_denom
        max_denom = abs(analytical g_i)
```

After the for loop above, the relative error is calculated using the final values of `max_denom` and `max_numerator`. The program then writes the current value of N and the error output to a `.csv` file, which is read by MATLAB code to save a log-log plot of the error output to `HW1_Error_Plot.png`.

Error Results

Here is `HW1_Error_Plot.png` with `step` = 10.

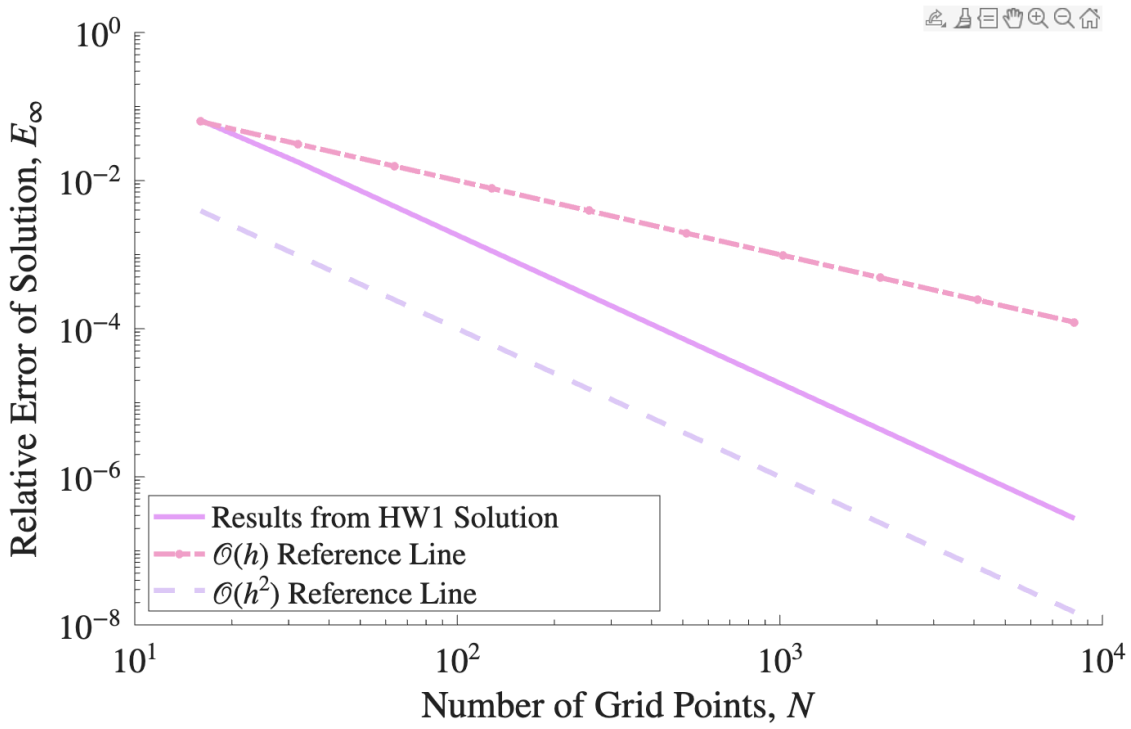


Figure 1: Plot of the relative error of our solution versus the number of grid points used.

One will note that the results of our solution converge with $\mathcal{O}(h^2)$, which is what we expect from a centered finite-difference method. Therefore, we can state that the code works as anticipated and produces a reliable result for appropriate values of N.

Code

HW1.cpp

```
#include <iostream>
#include <cmath>
#include <fstream> //for data writing purposes

//defines namespace as std so we don't have to repeat it
using namespace std;

int main(int argc, char *argv[])
{
 //------------------------------ Question 1 ----------------------------------------
 cout << "---------------------------" << endl;

 //user input for how many times they want to run the process (for how many N)
 const int stop = atoi(argv[1]) - 1;
  cout << "stop = " << stop << endl;

 // all file writing info referenced from
 // stackoverflow.com/questions/25201131/writing-csv-files-from-c

 ofstream file;
 file.open ("HW1_Error_Data.csv"); //opens/makes data file
 file << "N, Err_inf, \n"; //writes header

 //this for loops runs through the main code for different values of N, all multiples of 16
 for(int N = 16; N < (16*pow(2,stop) + 1); N *= 2){
  //print out result of N to confirm we are getting the N we expect
  cout << "N = " << N << endl;

  //define pi as an unchangeable constant
  const double pi = 4.0*atan(1.0); //this line was referenced from CPP Companion

  //define L as 2*pi
  double L = 2*pi;

  //define h as N/L
  double h = L/static_cast<double>(N);
  //cout << "Checking L/N = " << L << "/" << N << " = " << h << " = h" << endl;

//------------------------------ Question 2 ----------------------------------------


  //define grid points
  double x[N];

  //define function points
  double f[N];

  //define numerical derivative array
  double f_numerical[N];
```

```
  //define analytical derivative
  double f_analytical[N];

  //define 2pi/L so I don't have to write it 13 times
  double C = (2*pi)/L;

  for(int i=0; i < N; i++){
   x[i] = i*h; //declares the grid point by using h
   f[i] = exp(sin(C*x[i])); //declares function we wish to evaluate the derivative of
   f_analytical[i] = C*cos(C*x[i])*exp(sin(C*x[i])); //declares teh analytical solution to f
  }

//------------------------------ Question 3 ---------------------------------------

  for(int i = 0; i < N; i ++){
   if(i == 0){
    //if I'm on the left boundary, I want the last input of f
    f_numerical[i] = (f[i + 1] - f[N - 1])/(2*h);
   }
   else if (i == (N - 1)){
    //if I'm at my right boundary, I want my first input of f
    f_numerical[i] = (f[0] - f[i - 1])/(2*h);
   }
   else{
    //else,I just want my neighbors
    f_numerical[i] = (f[i + 1] - f[i - 1])/(2*h);
   }
  }

//------------------------------ Question 4 ---------------------------------------

  //declare a difference matrix for error calculation
  double diff_f_sol[N];

  //define starting max values, can use 0 as we are taking fabs
  double max_diff = 0;
  double max_analytical = 0;

  for(int i = 0; i < N; i++){
   //calculates the floating point absolute difference between our analytical and numerical solution
   diff_f_sol[i] = fabs(f_analytical[i] - f_numerical[i]);

   //checks to see if the current i-th entry of diff_f_sol is the maximum
   if(diff_f_sol[i] > max_diff){
    max_diff = diff_f_sol[i];
   }

   //determines maximum value of f_analytical
   if(fabs(f_analytical[i]) >  max_analytical){
    max_analytical = fabs(f_analytical[i]);
   }
```

```
  }

  double E_inf = max_diff/max_analytical;
  cout << "The global error, E_inf, is " << E_inf << endl;

  cout << "----------------------------" << endl;

  //writes current data to csv file
  file << N << "," << E_inf << ",\n";
 }

 file.close(); //closes csv file
 return 0;

//this ends main
}
```

Makefile

```
HW1: HW1.cpp
    g++ -o run_HW1 HW1.cpp
    ./run_HW1 10 #user input how many N to investigate

clean:
    rm -f run_HW1
    rm -f HW1_Error_Plot.csv
```

---

plot_error.m

```
plot_settings(1) %this turns on a file with settings i like for MATLAB plots
A = readmatrix("HW1_Error_Data.csv");
close all

N = A(:,1);
Err_inf = A(:,2);

f = figure(1);
theme(f,"light");
hold on
plot(N, (Err_inf), 'Color',[227/255,159/255,246/255])
plot(N, 1./N, '.-.', 'Color',[240/255,159/255,200/255])
plot(N, 1./(N.^2), '--', 'Color',[220/255,200/255,246/255])
set(gca,'YScale','log','XScale','log')
legend('Results from HW1 Solution', '$\cal{O}$$(h)$ Reference Line', '$\cal{O}$$(h^2)$
        Reference Line', 'Location','southwest')
ylabel("Relative Error of Solution, $E_{\infty}$")
xlabel("Number of Grid Points, $N$")
hold off

saveas(f, "HW1_Error_Plot.png", 'png')
```
